Control 1

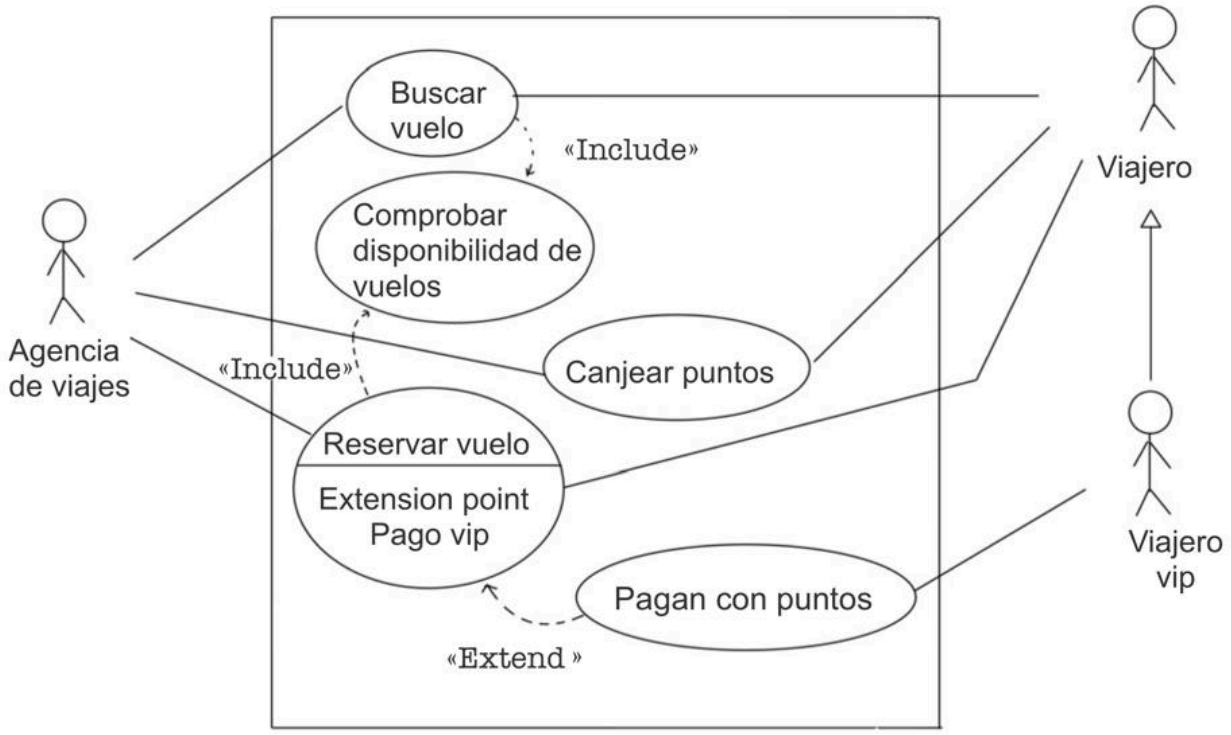
Juan Reche Aguilar

Ejercicio 1:

a) Indique los errores que se pueden observar en el diagrama y represente el correcto:

Los errores principales son el uso incorrecto de la flecha *extend*, siendo este un error directo al estar al revés, y las asociaciones duplicadas con los casos de uso de viajero vip, debido a que viajero vip hereda de la clase viajero no se deben volver a poner las asociaciones que tienen en común.
También se puede considerar errores las flechas de asociación de los agentes Agencia de Viajes y de Viajero con el caso de uso Comprobar disponibilidad de vuelos al estar este incluido en buscar vuelos y en reservar si no se va a utilizar de forma individual.
Por último, se ha reemplazado la asociación de viajero vip con reservar por la de viajero, bajo la idea de que no tendría sentido que solo viajero vip pudiese reservar vuelos.

El resultado final es el siguiente:



b) Usando la plantilla de descripción de actores, describa al actor “Agencia de viajes

Actor	Agencia de Viajes				
Descripción	Empresa autorizada que interactúa con el sistema para buscar y reservar vuelos para terceros				
Características	Suele tener asociadas condiciones comerciales, tarifas o permisos distintos a los de un viajero individual.				
Relaciones					
Referencias	CU-Buscar Vuelo CU-Reservar Vuelo CU-Canjear Puntos				
Autor		Fecha		Versión	

Atributos		
Nombre	Descripción	Tipo
Código Agencia	Identificador único de la agencia	Texto
Nombre Agencia	Nombre comercial de la agencia	Texto
Datos Contacto	Teléfono y email corporativos	Alfanumérico

Comentarios

c) Usando la plantilla de descripción extendida, realice la descripción del caso de uso “Buscar Vuelo”

Caso de Uso	Buscar Vuelo	CU-1
Actores	Agencia de Viajes (Principal), Viajero (Principal)	
Tipo	Primario, Esencial	

Referencias	CU-Comprobar Disponibilidad de Vuelos				
Precondición	El usuario que realiza la acción debe estar identificado				
Poscondición	El usuario recibe un listado de los vuelos que cumplen los criterios de búsqueda				
Autor		Fecha		Versión	

Propósito
Permitir encontrar vuelos que cumplan unos criterios establecidos.

Resumen
El actor solicita una búsqueda, introduce los parámetros del viaje que desea revisar y el sistema devuelve una lista de opciones.

Curso Normal (Básico)			
1	El Actor indica que desea buscar un vuelo	2	El Sistema solicita los datos del viaje.
3	El Actor proporciona la información solicitada.	4	El Sistema valida que los datos sean correctos
		5	Incluir: (Comprobar Disponibilidad de Vuelos).
		6	El Sistema muestra la lista de vuelos encontrados.

Cursos Alternos	
4a	Datos no válidos: 1. El Sistema informa que los datos introducidos son incorrectos. 2. Volvemos al paso 3.
6a	No hay vuelos: 1. El Sistema informa que no se han encontrado vuelos para los criterios seleccionados. 2. Se finaliza el curso de acción.

Otros datos			
Frecuencia esperada	10-20 al día	Rendimiento	Menos de 5 segundos
Importancia	Vital	Urgencia	Alta
Estado	En desarrollo	Estabilidad	Alta

Comentarios

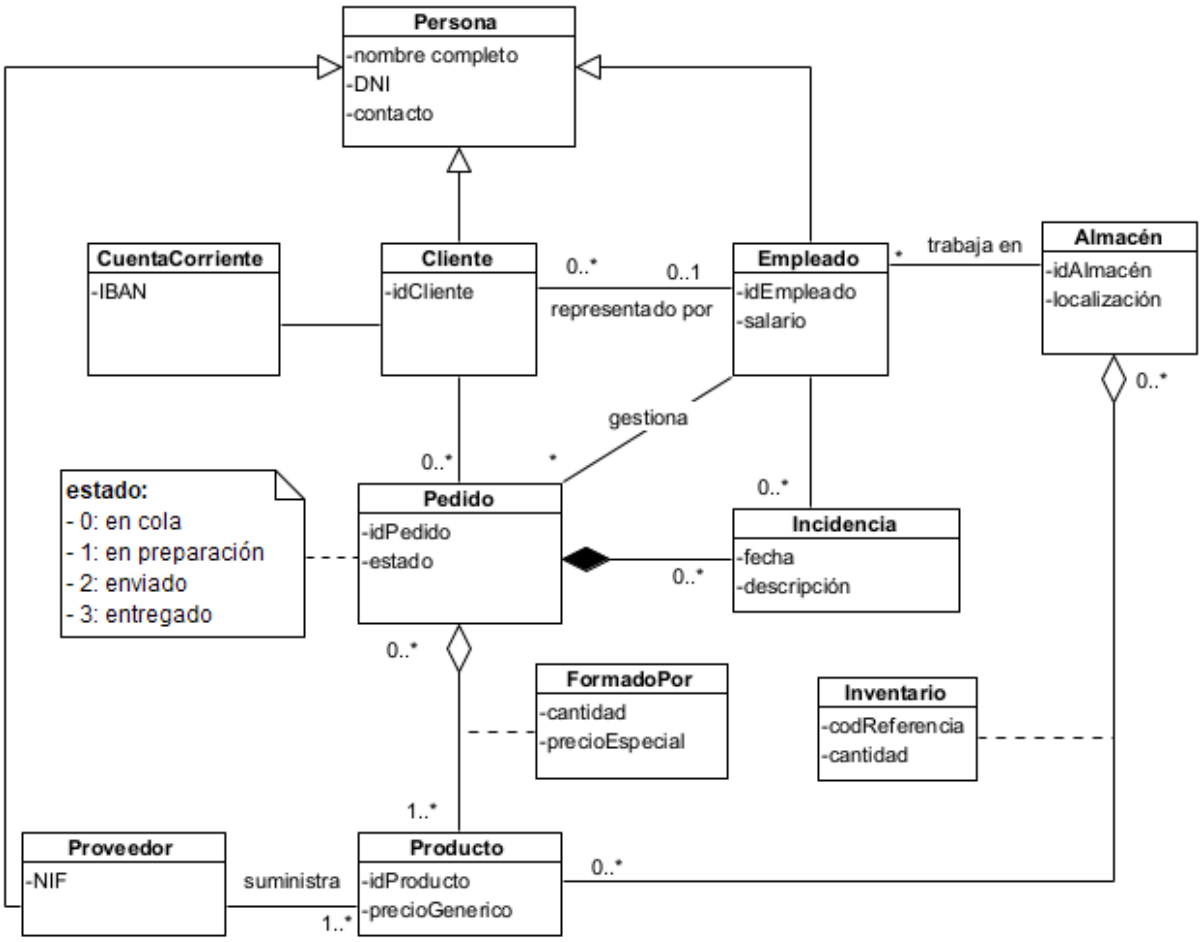
Ejercicio 2:

Explicación:

- Podemos encontrar las siguientes clases principales: Almacén, Empleado, Pedido, Cliente, Incidencia, Producto, Proveedor, CuentaCorriente.
- “En cada uno de los almacenes trabajan varios empleados”, un almacén recién creado o que no está en uso puede no tener empleados por lo que asignamos una relación de 0 o muchos (*).
- “labor principal es gestionar los pedidos”, se necesitaría un poco más de contexto para saber exactamente si pueden realizar otros trabajos pero entendiendo que si se opta por cero o muchos (*).
- Por las frases: “pedidos que realizan los clientes “ y “Normalmente, cada cliente tiene asignado a un empleado” se entiende una cardinalidad cero o muchos (0..*) y cero o uno (0...1) entendiendo que el sistema debe permitir tener clientes a la espera de que se le asigne un empleado.
- “cada pedido, se anotan las incidencias que los empleados observan” La relación entre Incidencia y Pedido es de composición, puesto que una incidencia carece de sentido si no tiene su pedido asociado.
- “fecha y la descripción de la incidencia”, nos obliga a poner como mínimo esos atributos.
- “información sobre el empleado que anota la incidencia” debe haber una relación entre empleado y incidencia, como un empleado puede poner varias y una incidencia solo es anotada por un empleado la relación es cero o muchas (0..*).
- “estado de su pedido”esto nos obliga a darle el atributo estado a la clase pedido.
- “Un pedido puede incluir diferentes productos” tenemos por tanto que establecer una relación de agregación además de una cardinalidad donde puede haber productos sin pedido (0..*) pero no pedidos si productos (1..).
- “dependiendo del cliente y de las cantidades que pida de cada producto, se le puede ofertar un precio diferente” debido a esta frase creamos la clase de asociación Formado donde incluir los atributos de cantidad y precioEspecial.
- “almacén se guarda información sobre los diferentes productos que se almacenan”, se establece una relación de agrupación.
- “precio y cantidad disponible” por esta frase se establece la clase de asociación inventario
- “el proveedor que los suministra” relación entre producto y proveedor donde se establece que se guarda registro si un proveedor tiene uno o varios de un producto.

- “localización de un producto en el almacén se realiza siempre por medio de un código” atributo obligatorio de producto.
- Por último, desde una perspectiva orientada a objetos se decide por crear la clase persona, siendo esta muy intuitiva y teniendo una relación de generalización de empleado, cliente y proveedor. Dejando así más limpia la lista de atributos de estas clases.

Mapa conceptual:



Ejercicio 3:

Nombre	**añadirListaEspera(idRestaurante, idCliente, fecha, hora)**
Responsabilidad	Crear una reserva en cola para la lista de espera del restaurante
Tipo	Sistema
Notas	
Excepciones	No existe un Restaurante con identificador idRestaurante. No existe un Cliente con identificador idCliente. Fecha y hora incorrectas (no están permitidas en el sistema ya sea por estar mal escritas, “16:65”, o representar un momento con el restaurante cerrado). No hay más espacio para reservas.
Salida	
Precondiciones	Existen el restaurante y el cliente, y la fecha y hora son válidas y posteriores al momento actual.
Poscondiciones	Se creó un objeto, llamémoslo nuevaReserva, de la clase ReservaEnCola debidamente inicializado. El atributo posición se aumentó en 1 con respecto al último objeto de la clase con ese idRestaurante. En caso de ser el primero se inicializa a 1. Fue creado un enlace entre nuevaReserva y el objeto de la clase Cliente identificado por idCliente. Fue creado un enlace entre nuevaReserva y el objeto de la clase ListaEspera que está asociado (mediante posee-una) al objeto de la clase Restaurante identificado por idRestaurante.

Nombre	**añadirPedido(idRestaurante, NumeroMesa, listaIdConsumición)**
Responsabilidad	Añadir un pedido a una mesa de un restaurante
Tipo	Sistema

Notas	
Excepciones	No existe un Restaurante con identificador idRestaurante. No existe una Mesa con atributo número igual a NumeroMesa asociada al Restaurante con identificador idRestaurante. Alguno de los objetos de la lista, listaIdConsumición, tiene algún atributo erróneo.
Salida	
Precondiciones	listaIdConsumición no está vacía.
Poscondiciones	Fue creado un objeto, nuevoPedido, de la clase Pedido. Fue creado un enlace entre nuevoPedido y el objeto de la clase Mesa identificado por NumeroMes. Para cada elemento de listaIdConsumición: Fue creado un enlace (de composición) entre nuevoPedido y el objeto de la clase Consumicion identificado por dicho elemento.